

APOSTILA DE COMPILADORES

Aula 4

Expressões Regulares e Autômatos Finitos

Disciplina: Compiladores e Interpretadores
Projeto integrador: compilador da linguagem MINIC

Material de apoio baseado no Capítulo 3 de
Compiladores: Princípios, Técnicas e Ferramentas
Aho, Lam, Sethi e Ullman

Aula 4 • Unidade de Análise Léxica

1. Objetivos de aprendizagem

Ao final desta aula, você deverá ser capaz de:

- explicar o que é uma linguagem regular e como ela é descrita por expressões regulares;
- usar união, concatenação, fecho de Kleene e fecho positivo para construir padrões;
- especificar tokens da MINIC com expressões regulares;
- diferenciar autômatos finitos determinísticos (AFD) e não determinísticos (AFND);
- acompanhar a conversão de um padrão em um reconhecedor;
- relacionar expressões regulares e autômatos à construção de analisadores léxicos.

2. Onde este conteúdo aparece no livro

A aula corresponde principalmente ao Capítulo 3, nas seções 3.3 (expressões regulares), 3.4 (diagramas de transição), 3.6 (autômatos finitos) e 3.7 (de expressões regulares para autômatos). Na edição consultada, o sumário indica o início de “Autômatos finitos” na p. 93 e “De expressões regulares a autômatos” na p. 97; a paginação do PDF pode não coincidir com a paginação impressa.

Ideia central

Expressões regulares especificam padrões; autômatos finitos reconhecem as cadeias desses padrões. Essa equivalência permite automatizar grande parte do analisador léxico.

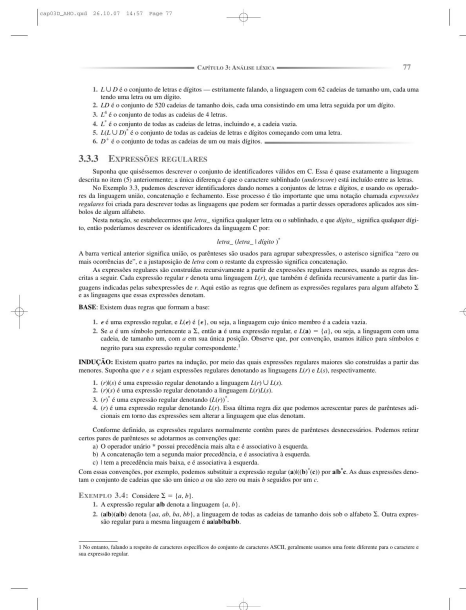


Figura de apoio: notação e exemplos de padrões no Capítulo 3, página impressa 77.

3. Linguagens, alfabetos e cadeias

Um **alfabeto** Σ é um conjunto finito de símbolos. Uma **cadeia** é uma sequência finita de símbolos de Σ . Uma **linguagem** é um conjunto de cadeias. No analisador léxico, trabalhamos com linguagens que descrevem classes de lexemas, como identificadores e números.

Exemplo

Considere $\Sigma = \{a, b\}$. Algumas cadeias são ϵ , a , b , ab e bba . A linguagem $L = \{a, ab, abb, abbb, \dots\}$ pode ser descrita pelo padrão ab^* .

4. Expressões regulares

A definição formal é recursiva. Para expressões regulares r e s :

Construção	Linguagem denotada	Exemplo
símbolo a	$\{a\}$	a
ϵ	$\{\epsilon\}$	ϵ
união $r \mid s$	$L(r) \cup L(s)$	$a b$
concatenação rs	$L(r)L(s)$	ab
fecho r^*	zero ou mais repetições	a^*
fecho positivo r^+	uma ou mais repetições	a^+

Precedência recomendada

1. fechos (* , $^+$, $?$); 2. concatenação; 3. união ($|$). Use parênteses para tornar a intenção explícita. Assim, $a|bc$ significa $a|(bc)$, enquanto $(a|b)c$ representa ac ou bc .

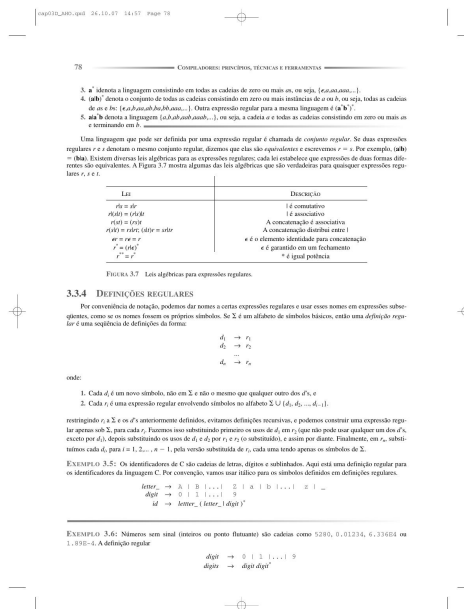


Figura de apoio: definições e propriedades de expressões regulares, Capítulo 3, p. 78.

5. Especificando tokens da MINIC

O scanner usa padrões para reconhecer lexemas. As palavras reservadas devem ser verificadas antes — ou distinguidas por uma tabela — para que `if` não seja classificado como identificador.

Token	Expressão regular sugerida	Exemplos
IDENT	<code>[A-Za-z_][A-Za-z0-9_]*</code>	contador, _x, soma2
INT	<code>[0-9]+</code>	0, 42, 2026
FLOAT	<code>[0-9]+\.[0-9]+</code>	3.14, 0.5
CHAR	<code>'([^\n\r\\])'</code>	'a', '\n'
OP_REL	<code>(== != < = < >)</code>	==, <=, !=
ESPACO	<code>[\t\n\r]+</code>	ignorado
COMENTARIO	<code>//.* ou /* ... */</code>	ignorado

Exercício guiado

Escreva uma expressão regular para cada classe: (a) identificador MINIC; (b) inteiro sem sinal; (c) número real; (d) operador de atribuição simples ou comparação; (e) cadeia delimitada por aspas duplas sem quebra de linha. Indique dois exemplos aceitos e dois rejeitados por padrão.

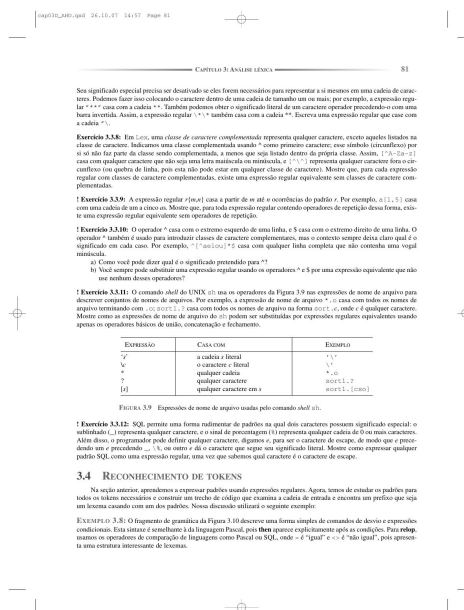


Figura de apoio: exemplos de expressões regulares e convenções de notação, Capítulo 3, p. 81.

6. De expressões regulares a diagramas de transição

Um diagrama de transição representa estados e movimentos condicionados pelos símbolos de entrada. Um estado inicial indica onde o reconhecimento começa; estados de aceitação indicam que o prefixo lido forma um lexema válido.

Exemplo: identificadores

Para o padrão $[A-Za-z_][A-Za-z0-9_]*$:

- o estado inicial exige uma letra ou sublinhado;
- depois do primeiro caractere, letras, dígitos e sublinhado podem repetir;
- o reconhecimento pode terminar em qualquer estado final após pelo menos um caractere;
- o scanner deve aplicar a regra de maior casamento (*maximal munch*).

Reconhecer não é classificar sozinho

O autômato informa se uma cadeia pertence ao padrão. A ação do scanner associa o reconhecimento a um token, registra atributos — por exemplo, valor numérico e posição — e decide o que fazer com palavras reservadas.

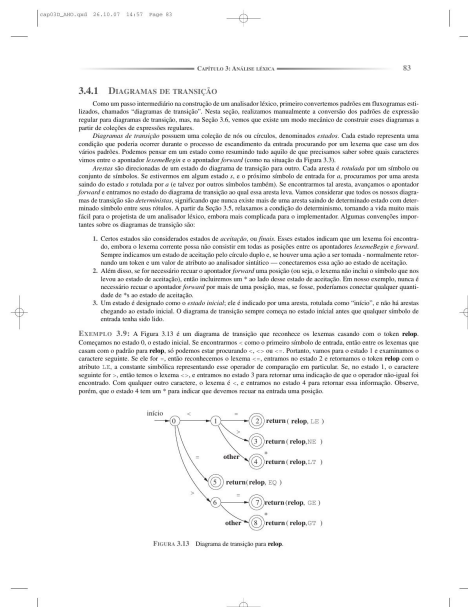


Figura de apoio: diagramas de transição usados na construção de analisadores léxicos, Capítulo 3, p. 83.

Atividade rápida

Desenhe um diagrama de transição para $a(b|c)^*$. Teste as cadeias ϵ , a , ab , acc , ba e acb . Marque quais são aceitas e em que ponto cada cadeia é rejeitada.

7. Autômatos finitos

Um autômato finito é um reconhecedor: recebe uma cadeia e responde aceita ou rejeita. O modelo possui estados, transições, um estado inicial e um conjunto de estados finais.

AFND e AFD

Característica	AFND	AFD
Transições	Pode haver várias transições para o mesmo símbolo de entrada.	Para cada estado e símbolo, há no máximo uma transição.
Execução	Pode explorar alternativas; aceita se algum caminho for bem-sucedido.	Existem apenas dois caminhos possíveis para cada cadeia de entrada.
Uso comum	Construção intermediária a partir de expressões regulares.	Reconhecimento eficiente no scanner.

Equivalência: AFNDs e AFDs reconhecem as mesmas linguagens regulares. A conversão AFND → AFD usa a construção de subconjuntos: cada estado do AFD representa um conjunto de estados possíveis do AFND.

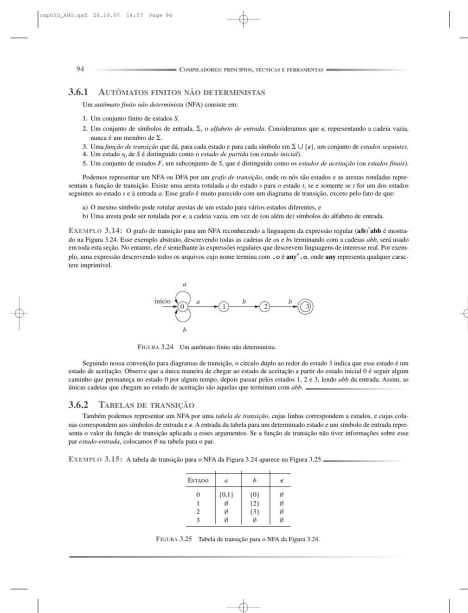


Figura de apoio: autômato finito não determinístico e seus componentes, Capítulo 3, p. 94.

Pergunta conceitual

Por que um AFD é conveniente para o scanner? Porque, ao ler cada caractere, o próximo estado é determinado sem precisar explorar vários caminhos.

8. Construção de subconjuntos

A conversão de um AFND em AFD mantém a linguagem reconhecida, mas transforma conjuntos de possibilidades em estados únicos. O procedimento básico é:

1. Comece pelo fecho- ϵ do estado inicial. 2. Para cada conjunto A e símbolo a , calcule $\text{move}(A, a)$. 3. Aplique novamente o fecho- ϵ . 4. Crie um novo estado para o conjunto obtido. 5. Marque como final todo conjunto que contenha um estado final do AFND.

Exemplo de leitura

Se, ao ler a , o AFND pode estar nos estados 2 ou 5, o AFD terá um estado representado por $\{2,5\}$. A partir desse estado, todas as transições são calculadas como um único movimento determinístico.

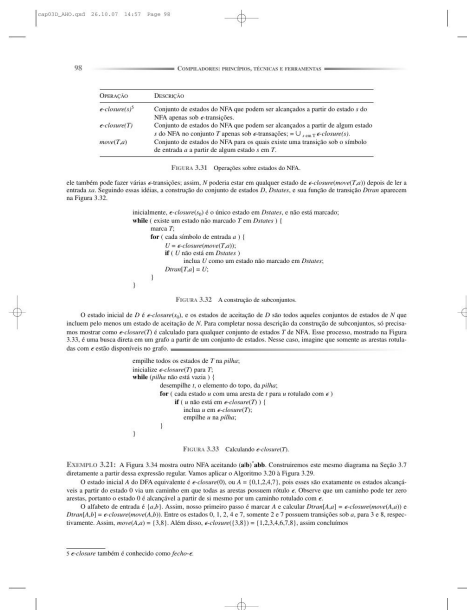


Figura de apoio: construção e simulação de subconjuntos na conversão para AFD, Capítulo 3, p. 98.

Conexão com o projeto MINIC

Na Etapa 1, a equipe pode implementar as regras diretamente em um gerador léxico ou construir um reconhecedor manual. Em ambos os casos, os padrões devem ser documentados e os casos de teste devem demonstrar o comportamento nos limites: lexemas vazios, prefixos válidos, símbolos inesperados e conflitos entre regras.

Exercício

Converta conceitualmente o padrão $(a|b)^*abb$ em um reconhecedor. Liste os estados necessários para acompanhar o progresso do sufixo “abb” e teste as cadeias abb, aabb, ab, abab e bbaabb.

9. De expressões regulares a AFND

Uma construção clássica associa a cada expressão regular um AFND. As regras são estruturais:

- para um símbolo, crie uma transição entre dois estados;
- para a união $r|s$, crie uma bifurcação ϵ para os autômatos de r e s ;
- para a concatenação rs , conecte o final de r ao início de s ;
- para r^* , crie caminhos ϵ que permitam zero ou mais repetições.

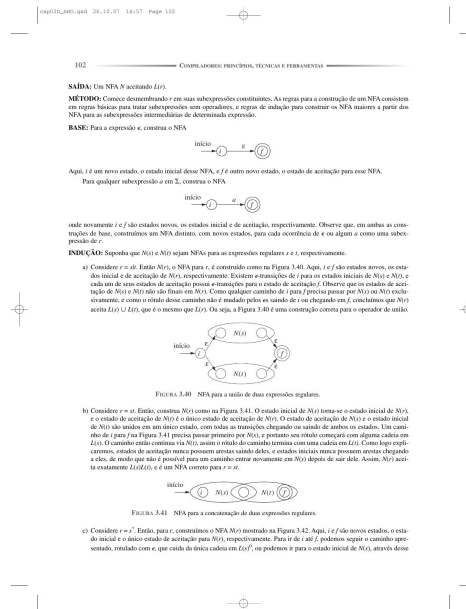


Figura de apoio: construção de autômatos a partir de subexpressões regulares, Capítulo 3, p. 102.

Por que usar uma construção intermediária?

Ela torna sistemática a transformação de padrões em reconhecedores. Depois, o AFND pode ser simulado diretamente ou convertido em AFD para obter uma execução eficiente.

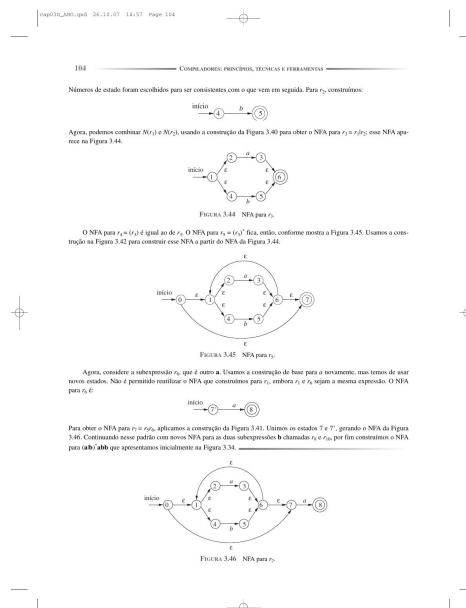


Figura de apoio: exemplos de composição de AFNDs para concatenação e repetição, Capítulo 3, p. 104.

Resumo da aula

- expressões regulares descrevem linguagens regulares;

- AFNDs são flexíveis para construção;
- AFDs oferecem execução determinística;
- a conversão entre essas representações fundamenta geradores de analisadores léxicos;
- o scanner associa reconhecimento a tokens e ações semânticas.

10. Laboratório e preparação da Etapa 1

Atividade integrada

Projete o conjunto de tokens do MINIC e produza uma tabela com: nome do token, padrão, exemplos válidos, exemplos inválidos, prioridade e ação associada. Depois, teste os padrões em um programa que contenha declarações, expressões, comentários e erros.

Item de verificação	Pergunta
Cobertura	Todos os tokens obrigatórios da MINIC foram especificados?
Prioridade	Palavras reservadas são distinguidas de identificadores?
Maior casamento	Operadores como =, ==, <, <= são resolvidos corretamente?
Posição	Cada erro informa linha e coluna?
Robustez	Comentários e literais não terminados são rejeitados corretamente?
Testes	Há casos positivos, negativos e de fronteira?

Exercícios de fixação

1. Descreva a linguagem de $(ab|c)^*$. 2. Escreva um padrão para identificadores MINIC. 3. Explique a diferença entre AFND e AFD. 4. Por que o fecho- ϵ é necessário? 5. Dê um exemplo de duas regras léxicas que exigem prioridade. 6. Construa um reconhecedor para inteiros positivos sem zeros à esquerda.

Desafio

Projete padrões para comentários de linha e de bloco. Explique como sua implementação detectará um comentário de bloco sem fechamento e qual diagnóstico será emitido.

Referência principal

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. Capítulo 3 — Análise léxica, especialmente as seções 3.3, 3.4, 3.6 e 3.7. As imagens reproduzidas nesta apostila são excertos de apoio didático do exemplar fornecido pelo usuário; consulte o livro para o contexto completo, definições formais e exercícios originais.